JG REPUESTOS

Entrega proyecto 1 · Escenario 3

Docker, Docker Compose y GitHub Actions

Material académico preparado con resultados reales y validaciones pendientes identificadas.

Datos de estudiante, institución, asignatura y docente: completar antes de entregar.

Fecha límite: 10 de noviembre de 2026 · 23:55 · America/Bogota

Estado: Docker validado localmente; ejecución remota de GitHub pendiente.

Portada

Proyecto: Sistema web de gestión comercial e inventario JG Repuestos.

Actividad: Contenedores Docker, Docker Compose y GitHub Actions.

Estudiante, institución, asignatura y docente: completar con los datos reales antes de entregar.

Fecha de preparación: 8 de noviembre de 2026. Actualización y validación Docker: 9 de noviembre de 2026. Zona horaria: America/Bogota.

Fecha límite: martes 10 de noviembre de 2026 a las 23:55.

Repositorio GitHub: https://github.com/JesusGalvis05/Repuestos_SAS

El remoto origin está configurado. La consulta remota requiere autenticación y la API pública respondió 404; no se pudo confirmar desde esta sesión si los commits ya están publicados ni si Actions terminó en verde.

Estado: construcción, salud, migraciones, comunicación, persistencia y Pytest aprobados en Docker local. Verificación de los commits publicados y de la ejecución remota de GitHub pendiente de autenticación.

1. Introducción

JG Repuestos es un proyecto de gestión comercial para un negocio colombiano de repuestos automotrices. Su implementación incremental comienza con autenticación, usuarios, permisos y sesiones. La actividad académica adapta esa base a dos contenedores comunicados y a un flujo de integración continua reproducible.

Esta entrega se limita a DevOps y a la Fase 1 existente. No incorpora productos, compras ni otras funciones de la Fase 2. Distingue expresamente las verificaciones realizadas de las que requieren infraestructura todavía no disponible.

2. Objetivo general

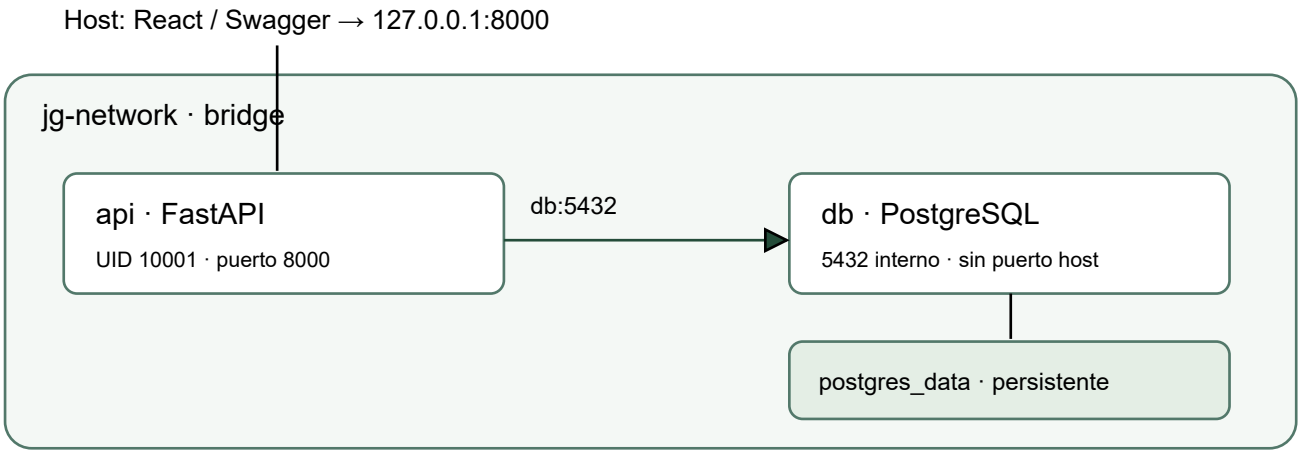
Preparar y verificar una solución de dos servicios Docker para FastAPI y PostgreSQL, con comunicación mediante DNS interno, persistencia, configuración segura y un pipeline automatizado que genere evidencias reales.

3. Descripción de la aplicación

El backend utiliza Python, FastAPI, SQLAlchemy 2.0, Pydantic v2 y Alembic; la aplicación mantiene un monolito modular y una única aplicación ASGI desplegable. El frontend React y TypeScript permanece en su carpeta y se ejecuta desde el host durante esta demostración. El Compose académico no contiene un servicio frontend.

La API ofrece autenticación, control de sesiones, autorización RBAC y gestión de usuarios. Las contraseñas se almacenan con Argon2id. Los endpoints de gestión exigen autorización; la actividad no añade rutas públicas de escritura.

4. Arquitectura y red Docker



Existen exactamente dos servicios de ejecución: api y db. La red bridge tiene el nombre explícito jg-network. Sólo api publica el puerto 8000, ligado a la interfaz de bucle local. PostgreSQL no publica puertos al host; su 5432 es interno. El volumen postgres_data conserva el directorio de datos cuando se recrean los contenedores sin eliminarlo.

La API recibe POSTGRES_HOST=db y POSTGRES_PORT=5432. Construye una URL con SQLAlchemy.URL.create usando usuario, contraseña y nombre de base provenientes del entorno. Esto codifica correctamente caracteres especiales. DOCKER_RUNTIME impide arrancar con localhost, IPs fijas u otro puerto como destino de PostgreSQL.

5. Contenedor de aplicación

backend/Dockerfile utiliza la imagen oficial python:3.12.16-slim-bookworm. Copia primero requirements.lock para aprovechar la caché de dependencias; instala con pip sin caché y después copia el código y las migraciones. La etapa final runtime no contiene Pytest ni las pruebas; la etapa opcional test agrega esas herramientas.

El proceso se ejecuta con UID y GID 10001, sin privilegios de root, y expone 8000. Su comando python -m app.deploy ejecuta Alembic upgrade head antes de iniciar Uvicorn. Un fallo de migración impide servir la API. No se usa create_all para evolucionar el esquema ni se crea un tercer servicio permanente.

La imagen de PostgreSQL es postgres:17.11-bookworm. Ambas etiquetas se contrastaron con los manifiestos de Docker Official Images. El 9 de noviembre se descargó PostgreSQL y se construyeron las imágenes de ejecución y pruebas con éxito. El log de construcción conserva el identificador SHA-256 real de la imagen obtenida.

6. Docker Compose

El archivo canónico es docker-compose.yml. Se retiró la configuración anterior de cuatro servicios para evitar que Compose seleccione accidentalmente otro archivo. api depende de db mediante condition: service_healthy. PostgreSQL usa pg_isready con el usuario y la base configurados; api consulta su endpoint ready.

Ambos servicios usan restart: unless-stopped. api restringe privilegios nuevos y elimina capacidades Linux adicionales. Los secretos se generan en .env y no se incluyen en las imágenes ni se versionan. .env.example contiene campos vacíos de secreto; scripts/configure.py genera valores aleatorios sin imprimirlos.

Estos comandos forman parte de la validación reproducible:

```
docker compose config
docker compose build
docker compose up -d
docker compose ps
docker ps
docker network inspect jg-network
docker compose logs api
docker compose logs db
```

El recolector invoca config en formato JSON y guarda una versión saneada. Un config manual imprime secretos interpolados y no debe capturarse sin ocultarlos. Los comandos up del recolector añaden --wait para comprobar salud.

7. Demostración de escritura y lectura

```
docker compose exec -T api python -m app.diagnostics roundtrip
```

El diagnóstico usa exactamente la misma configuración y fábrica de sesiones que la API. Inserta un usuario UUID inactivo, sin roles, con nombre y correo marcados como demostración; su contraseña aleatoria se descarta. Confirma la transacción, consulta el registro en una sesión nueva y valida id, correo, nombre y estado. También consulta current_database e inet_server_port para identificar la conexión.

La limpieza elimina sólo el UUID creado y exige simultáneamente correo derivado, nombre de demostración, estado inactivo y ausencia de roles. Una prueba comprueba que no puede eliminar un administrador normal. No se vacían tablas ni se borran datos operacionales. No se añade un endpoint público para esta prueba.

La demostración se ejecutó dentro de api contra PostgreSQL db. El JSON real confirma host db, puerto configurado 5432, puerto consultado al servidor 5432 y cleanup=passed. También se aprobaron las pruebas de integración dentro de api.

8. Prueba de persistencia

scripts/docker_evidence.py crea otro registro marcado, conserva su UUID, ejecuta docker compose down sin -v, vuelve a levantar ambos contenedores, lee el mismo UUID y elimina únicamente ese registro. El volumen persiste entre ambos arranques. Si la limpieza falla, el informe guarda pending_demo_id para una recuperación limitada a ese registro. La secuencia se ejecutó con éxito: se recuperó el UUID d9773fcd-b92d-4d3d-8e7d-2cae4e7ccb34 después de recrear ambos contenedores y se eliminó únicamente ese registro de demostración. Los logs originales conservan creación, parada, arranque, lectura y limpieza.

9. Integración continua

.github/workflows/docker-ci.yml se activa por push y pull_request, con permisos de lectura del repositorio. Obtiene el código, prepara Buildx y genera credenciales efímeras enmascaradas. El recolector construye la imagen, descarga PostgreSQL, valida Compose, levanta dos contenedores, espera salud y ejecuta comunicación y persistencia. Pytest se ejecuta dentro de api usando la etapa de pruebas y la misma base db; no se agrega un tercer servicio. Un job separado prueba y compila React.

Los artefactos contienen salidas saneadas y códigos de retorno. El apagado usa if: always(). No se publican imágenes en registros externos. La configuración del workflow existe y origin está configurado; su ejecución aprobada en GitHub no está verificada porque la consulta requiere autenticación en este entorno.

10. Resultados reales y límites

Verificación	Resultado real
Revisión estática de archivos	18 comprobaciones aprobadas; no sustituye Docker
Suite backend con PostgreSQL nativo	31 aprobadas, 1 omitida exclusiva de Docker
Suite backend dentro de api contra db	32 aprobadas, sin omisiones
Prueba temporal de escritura/lectura/limpieza	Aprobada dentro de Docker mediante db:5432
Protección contra limpieza de usuarios normales	Aprobada también dentro de Docker
Compose oficial portátil 2.39.4	SHA-256 contrastado y config validado con código 0
Docker CLI y motor	Docker Engine 29.9.2; Compose 5.6.1; motor Linux disponible
Construcción y migraciones	Aprobadas; revisión 0001_identity y alembic check sin diferencias
Red y salud	api y db saludables en jg-network; sólo 127.0.0.1:8000 publicado
Persistencia	Mismo UUID recuperado tras down/up conservando el volumen
Usuario y restauración	UID 10001 comprobado; imagen runtime restaurada tras las pruebas
Repositorio remoto y pipeline aprobado	Pendientes de GitHub

El aviso de deprecación httpx de Starlette no provocó fallos en Pytest. Los logs de PostgreSQL nativo están en evidencias/pytest-local.txt. Las pruebas Docker, logs de comandos y resultados del 9 de noviembre se conservan en evidencias/ejecuciones/20261109-193746/. Los informes blocked del 8 de noviembre se conservan como historial del entorno anterior, ya superado al instalar Docker.

Se descargó Compose standalone oficial únicamente en .local/tools, excluido del repositorio. Se verificó su SHA-256 y se ejecutaron version, config --quiet y config --format json con éxito. Las evidencias compose-validacion-real.json y compose-config-real.json documentan la validación y ocultan secretos. Disponer de este parser no instala ni sustituye Docker Engine.

11. Evidencias incorporadas y pendientes

El PDF incorpora extractos reales de build, compose up, docker ps, network inspect, escritura/lectura, recreación con persistencia, migraciones y Pytest. Los archivos completos saneados están incluidos en el repositorio. Se incorporan capturas reales del navegador para ready y Swagger, con URL y fecha en capturas-http.json. Siguen pendientes los PNG de consola, GitHub Actions remoto aprobado y repositorio publicado. La lista precisa de archivos y comandos se conserva en capturas/README.md. No se han generado imágenes que imiten una consola ni resultados de CI inexistentes.

Antes de entregar se deben completar los datos de portada, confirmar el acceso al repositorio enlazado e incorporar las capturas y el pipeline remoto aprobado. La demostración Docker ya está ejecutada y documentada. La matriz de ocho criterios y sus 150 puntos está en rubrica.md.

12. Conclusiones

La configuración preparada reduce el entorno a dos servicios y establece un contrato verificable de conexión db:5432, migraciones previas al servidor y datos persistentes en un volumen. La demostración privada conserva la autenticación y limita sus cambios a usuarios temporales inactivos.

Las pruebas Docker aportan evidencia real de la comunicación y limpieza segura, además de persistencia tras recrear los contenedores. La validación técnica local de Docker está aprobada. La aceptación del pipeline remoto y de la entrega académica continúa pendiente del repositorio publicado, capturas y datos de portada. No se avanza a la Fase 2 ni se atribuyen puntos de evaluación sin revisión docente.

13. Referencias — APA 7

Docker, Inc. (s. f.). Control startup and shutdown order in Compose. Docker Docs. Recuperado el 8 de noviembre de 2026, de https://docs.docker.com/compose/how-tos/startup-order/

Docker, Inc. (s. f.). Docker Build GitHub Actions. Docker Docs. Recuperado el 8 de noviembre de 2026, de https://docs.docker.com/build/ci/github-actions/

Docker Official Images. (s. f.). Python: Official image metadata [Código fuente]. GitHub. Recuperado el 8 de noviembre de 2026, de https://github.com/docker-library/official-images/blob/master/library/python

Docker Official Images. (s. f.). PostgreSQL: Official image metadata [Código fuente]. GitHub. Recuperado el 8 de noviembre de 2026, de https://github.com/docker-library/official-images/blob/master/library/postgres

SQLAlchemy. (s. f.). Session basics. SQLAlchemy 2.0 Documentation. Recuperado el 8 de noviembre de 2026, de https://docs.sqlalchemy.org/en/20/orm/session_basics.html

Anexo: salida real de Pytest local

```
$ python -m pytest -q -p no:cacheprovider
.......s.......................                                         [100%]
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
31 passed, 1 skipped, 1 warning in 12.20s

Exit code: 0
```

Anexo: ejecuciones reales Docker

Ejecución: 20261109-193746. Los bloques siguientes son extractos de salidas originales saneadas; los archivos completos están en evidencias/ejecuciones/20261109-193746/.

Construcción de la imagen

```
$ docker build --target runtime --tag jg-repuestos-api:academic backend
[Extracto: se omiten líneas intermedias; ver archivo original]
#10 [base 5/8] COPY app ./app
#10 DONE 0.0s

#11 [base 6/8] COPY migrations ./migrations
#11 DONE 0.0s

#12 [base 7/8] COPY alembic.ini ./
#12 DONE 0.0s

#13 [base 8/8] RUN groupadd --gid 10001 jg && useradd --create-home --uid 10001 --gid 10001 jg
#13 DONE 0.1s

#14 exporting to image
#14 exporting layers
#14 exporting layers 0.3s done
#14 writing image sha256:3ae545100fd4cf15aa91d0eb4f28e83fe1a1dae315664e9a3b170d6ab67d76f7 done
#14 naming to docker.io/library/jg-repuestos-api:academic done
#14 DONE 0.3s

Exit code: 0
```

Arranque y salud de los servicios

```
$ docker compose up -d --wait --wait-timeout 180
[Extracto: se omiten líneas intermedias; ver archivo original]
 Network jg-network Creating
 Network jg-network Creating
 Volume jg-repuestos_postgres_data Creating
 Volume jg-repuestos_postgres_data Creating
 Volume jg-repuestos_postgres_data Created
 Volume jg-repuestos_postgres_data Created
 Network jg-network Created
 Network jg-network Created
 Container jg-repuestos-db-1 Creating
 Container jg-repuestos-db-1 Created
 Container jg-repuestos-api-1 Creating
 Container jg-repuestos-api-1 Created
 Container jg-repuestos-db-1 Starting
 Container jg-repuestos-db-1 Started
 Container jg-repuestos-db-1 Waiting
 Container jg-repuestos-db-1 Healthy
 Container jg-repuestos-api-1 Starting
 Container jg-repuestos-api-1 Started
 Container jg-repuestos-db-1 Waiting
 Container jg-repuestos-api-1 Waiting
 Container jg-repuestos-db-1 Healthy
 Container jg-repuestos-api-1 Healthy

Exit code: 0
```

Dos servicios y puertos

```
$ docker compose ps
NAME                IMAGE                       COMMAND                  SERVICE   CREATED
          STATUS                    PORTS
jg-repuestos-api-1   jg-repuestos-api:academic   "python -m app.deploy"   api       11 seconds
ago   Up 5 seconds (healthy)    127.0.0.1:8000->8000/tcp
jg-repuestos-db-1    postgres:17.11-bookworm     "docker-entrypoint.s…"   db        11 seconds
ago   Up 11 seconds (healthy)   5432/tcp

Exit code: 0
```

Inspección de jg-network

```
$ docker network inspect jg-network
[
    {
        "Name": "jg-network",
        "Id": "6d8247de84f90372efe1bd36508af8b536e06cfef9c970a40744efce2b623163",
        "Created": "2026-11-10T00:38:48.355577809Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {
            "com.docker.compose.config-hash":
"f1686f33d5a754be28a8566f587cec04f8a5e237744f124b9930fb674e91c748",
            "com.docker.compose.network": "jg-network",
            "com.docker.compose.project": "jg-repuestos",
            "com.docker.compose.version": "5.6.1"
        },
        "Containers": {
            "4e7d215d7c3c3b047f1e8f27ad648157cba9469b58bb977a5a1f179ffce75c4e": {
                "Name": "jg-repuestos-api-1",
                "EndpointID":
"77614f3f379c833dd434dbc0e29e9e73625abc4f8b28a90d2b164436bb29c231",
                "MacAddress": "0a:62:b8:a8:ca:f9",
                "IPv4Address": "172.18.0.3/16",
                "IPv6Address": ""
            },
            "ca47a6bbe30889fdd483e33a675c558a271a70b5cd8ea2e49a32fcf74027dce6": {
                "Name": "jg-repuestos-db-1",
                "EndpointID":
"d8492f39d1f76861dc7758a7df29fe7f1b2d24f8af9ca7bfbdef5ce49e95ecd0",
                "MacAddress": "8e:20:94:f1:9a:8b",
                "IPv4Address": "172.18.0.2/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.18.0.0/16": {
                        "IPsInUse": 5,
                        "DynamicIPsAvailable": 65531
                    }
                }
            }
        }
    }
]

Exit code: 0
```

Usuario no privilegiado

```
$ docker compose exec -T api id -u
10001

Exit code: 0
```

Revisión Alembic

```
$ docker compose exec -T api alembic current
0001_identity (head)
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.

Exit code: 0
```

Correspondencia entre esquema y modelos

```
$ docker compose exec -T api alembic check
No new upgrade operations detected.
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.schemas
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.tables
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.types
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.constraints
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.defaults
INFO  [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.comments

Exit code: 0
```

Escritura, lectura y limpieza

```
$ docker compose exec -T api python -m app.diagnostics roundtrip
{"status": "passed", "operation": "write-read-cleanup", "id":
"ccb7789b-552d-4ddb-8720-a89de00a95ac", "host": "db", "configured_port": 5432, "server_port":
5432, "database": "jg_repuestos", "verified": ["id", "email", "full_name", "inactive",
"no_roles"], "cleanup": "passed"}

Exit code: 0
```

Crear registro para persistencia

```
$ docker compose exec -T api python -m app.diagnostics create
{"status": "passed", "operation": "create", "id": "d9773fcd-b92d-4d3d-8e7d-2cae4e7ccb34"}

Exit code: 0
```

Recrear sin eliminar el volumen: parada

```
$ docker compose down
 Container jg-repuestos-api-1 Stopping
 Container jg-repuestos-api-1 Stopped
 Container jg-repuestos-api-1 Removing
 Container jg-repuestos-api-1 Removed
 Container jg-repuestos-db-1 Stopping
 Container jg-repuestos-db-1 Stopped
 Container jg-repuestos-db-1 Removing
 Container jg-repuestos-db-1 Removed
 Network jg-network Removing
 Network jg-network Removed

Exit code: 0
```

Recrear sin eliminar el volumen: arranque

```
$ docker compose up -d --wait --wait-timeout 180
[Extracto: se omiten líneas intermedias; ver archivo original]
 Network jg-network Creating
 Network jg-network Creating
 Network jg-network Created
 Network jg-network Created
 Container jg-repuestos-db-1 Creating
 Container jg-repuestos-db-1 Created
 Container jg-repuestos-api-1 Creating
 Container jg-repuestos-api-1 Created
 Container jg-repuestos-db-1 Starting
 Container jg-repuestos-db-1 Started
 Container jg-repuestos-db-1 Waiting
 Container jg-repuestos-db-1 Healthy
 Container jg-repuestos-api-1 Starting
 Container jg-repuestos-api-1 Started
 Container jg-repuestos-db-1 Waiting
 Container jg-repuestos-api-1 Waiting
 Container jg-repuestos-db-1 Healthy
 Container jg-repuestos-api-1 Healthy

Exit code: 0
```

Mismo UUID recuperado

```
$ docker compose exec -T api python -m app.diagnostics read --id
d9773fcd-b92d-4d3d-8e7d-2cae4e7ccb34
{"status": "passed", "operation": "read", "id": "d9773fcd-b92d-4d3d-8e7d-2cae4e7ccb34", "host":
"db", "configured_port": 5432, "server_port": 5432, "database": "jg_repuestos", "verified":
["id", "email", "full_name", "inactive", "no_roles"]}

Exit code: 0
```

Limpieza del registro temporal

```
$ docker compose exec -T api python -m app.diagnostics delete --id
d9773fcd-b92d-4d3d-8e7d-2cae4e7ccb34
{"status": "passed", "operation": "delete", "id": "d9773fcd-b92d-4d3d-8e7d-2cae4e7ccb34"}

Exit code: 0
```

Pytest dentro de api con PostgreSQL db

```
$ docker compose exec -T -e RUN_DOCKER_COMMUNICATION=1 api python -m pytest -q -p
no:cacheprovider
.............................                                            [100%]
============================== warnings summary ===============================
../usr/local/lib/python3.12/site-packages/fastapi/testclient.py:1
  /usr/local/lib/python3.12/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning:
Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
32 passed, 1 warning in 4.36s

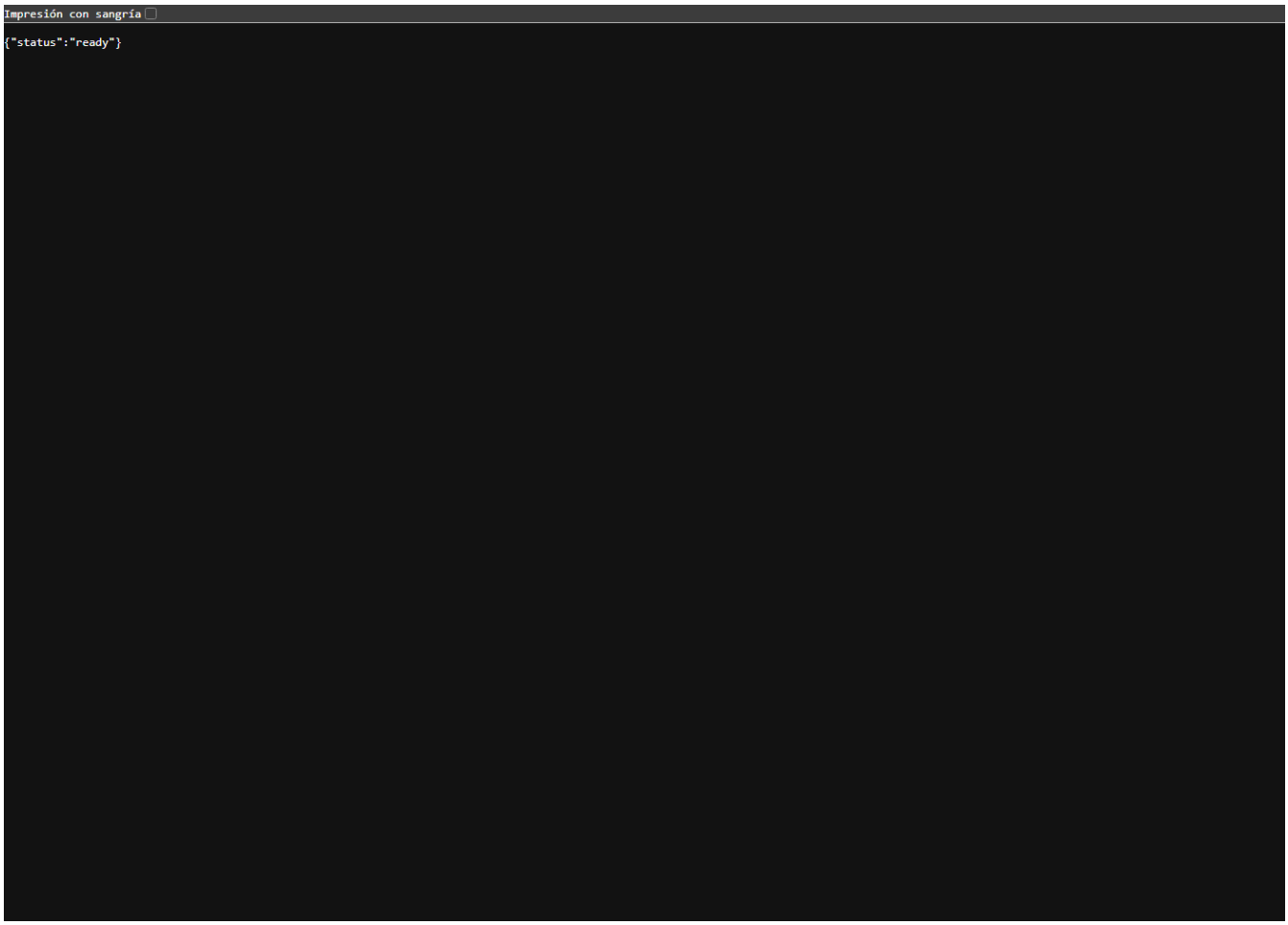
Exit code: 0
```

Estado final tras restaurar runtime

```
$ docker compose ps
NAME                IMAGE                       COMMAND                  SERVICE   CREATED
             STATUS                         PORTS
jg-repuestos-api-1   jg-repuestos-api:academic   "python -m app.deploy"   api       7 seconds
ago        Up 5 seconds (healthy)        127.0.0.1:8000->8000/tcp
jg-repuestos-db-1    postgres:17.11-bookworm     "docker-entrypoint.s…"   db        About a
minute ago   Up About a minute (healthy)   5432/tcp

Exit code: 0
```

11-api-ready.png



12-swagger.png

